

```
In [1]: %matplotlib inline

import jax.numpy as jnp
from jax import value_and_grad
from jax import random
from scipy.optimize import minimize

import matplotlib.pyplot as plt
import seaborn as snb

from mpl_toolkits.axes_grid1 import make_axes_locatable

def add_colorbar(im, fig, ax):
    divider = make_axes_locatable(ax)
    cax = divider.append_axes('right', size='5%', pad=0.05)
    fig.colorbar(im, cax=cax, orientation='vertical')

# we want to use 64 bit floating precision
import jax
jax.config.update("jax_enable_x64", True)

snb.set_style('darkgrid')
snb.set_theme(font_scale=1.25)
```

02477 Bayesian Machine Learning: Exercise 5

The purpose of this exercise is to

- learn how to sample from a multivariate Gaussian distribution
- become familiar with covariance functions, especially stationary covariance functions such as the squared exponential and Matérn family and to understand their properties and hyperparameters
- become familiar with Gaussian processes and learn to use them as prior distribution for latent functions in regression settings

Content

- Part 1: Sampling from a multivariate Gaussian distribution
- Part 2: Stationary covariance functions
- Part 3: Non-linear regression using Gaussian processes
- Part 4: Hyperparameter optimization using the marginal likelihood
- Part 5: Analysing the bike sharing data set

Note: The exercise contains several **discussion questions**, which are questions, where are supposed to actively experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs. Instead of proving the full description for every discussion question, we simply tag it with: **[Discussion question]** after the question.

Introduction

When we worked with parametric models such as linear regression and logistic regression, we assigned prior distributions on the weights $\mathbf{w}$ of the models. In contrast, we can think of Gaussian processes as prior distributions directly on the function space

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$$

where $f: \mathbb{R}^D \rightarrow \mathbb{R}$ and $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$, and $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))]$ are the **mean function** and **covariance function**, respectively, for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}$.

A Gaussian process is completely specified by its mean and covariance function. Most often we take the mean function to be zero, $m(\mathbf{x}) = 0$, because we don't have any prior information on the mean function:

$$f(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}')), \quad (1)$$

The **covariance function** (or kernel function) $k(\mathbf{x}, \mathbf{x}')$ thus completely determines the a priori behavior of the latent function f . Specifically, the covariance function determines the covariance of the latent function f when evaluated at two different inputs, i.e.

$$\text{cov}[f(\mathbf{x}), f(\mathbf{x}')] = k(\mathbf{x}, \mathbf{x}'). \quad (2)$$

The **squared exponential** covariance function is given by

$$k_{\text{SE}}(\mathbf{x}_n, \mathbf{x}_m) = \kappa^2 \exp\left(-\frac{\|\mathbf{x}_n - \mathbf{x}_m\|_2^2}{2\ell^2}\right), \quad (3)$$

where $\kappa > 0$ and $\ell > 0$ are hyperparameters of the kernel. This specific covariance function is perhaps the most common covariance function used in statistics and machine learning. It is also known as the radial basis function kernel, the gaussian kernel, or the exponentiated quadratic kernel. We will also study two covariance functions from the Matérn family:

$$k_{\text{Matern12}}(\mathbf{x}_n, \mathbf{x}_m) = \kappa^2 \exp\left(-\frac{\|\mathbf{x}_n - \mathbf{x}_m\|}{\ell}\right)$$

$$k_{\text{Matern32}}(\mathbf{x}_n, \mathbf{x}_m) = \kappa^2 \left(1 + \frac{\sqrt{3}\|\mathbf{x}_n - \mathbf{x}_m\|}{\ell}\right) \exp\left(-\frac{\sqrt{3}\|\mathbf{x}_n - \mathbf{x}_m\|}{\ell}\right).$$

Part 1: Sampling from a multivariate Gaussian distribution

To become familiar with covariance functions and understand its properties, we will generate and visualize samples from Gaussian process distributions. Therefore, our first order of business will be to implement a function for generating samples from a Gaussian process prior.

Let $\mathbf{X} = \{\mathbf{x}_n\}_{n=1}^N$ be a set of points in the real line, i.e. $\mathbf{x}_n \in \mathbb{R}^D$, and let $f: \mathbb{R}^D \rightarrow \mathbb{R}$ be a function. Let $f_n \equiv f(x_n) \in \mathbb{R}$ be the value of the function f evaluated at $x_n \in \mathbb{R}$. Furthermore, let $\mathbf{f} \equiv [f_1, f_2, \dots, f_N] \in \mathbb{R}^{N \times 1}$ be the vector of function values for each of the points in $\mathbf{X}$, then the Gaussian process prior for $\mathbf{f}$ becomes

$$\mathbf{f} \sim \mathcal{N}(\mathbf{0}, \mathbf{K}), \quad (4)$$

where $\mathbf{K} \in \mathbb{R}^{N \times N}$ is the kernel matrix satisfying $\mathbf{K}_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$, where $\mathbf{x}_n$ refers to the n 'th input point, i.e. the i 'th entry of $\mathbf{X}$.

The **Cholesky decomposition** is sometimes referred to as *matrix square root* and it is really handy matrix decomposition for sampling from a multivariate Gaussian distribution. The Cholesky decomposition of the kernel matrix is given by $\mathbf{K} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L} \in \mathbb{R}^{N \times N}$ is a lower triangular matrix.

The idea is to generate samples from standardized Gaussian distributions, and then transform those samples into the desired distribution. Let $\mathbf{z} \in \mathbb{R}^{N \times 1}$ be a sample from a standardized multivariate Gaussian distribution, i.e. $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N)$, where $\mathbf{I}_N$ is the identity matrix of size $N \times N$.

Task 1.1: Show that if $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_N)$ and $\mathbf{m} \in \mathbb{R}^N$, then $\mathbf{f} = \mathbf{L}\mathbf{z} + \mathbf{m} \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$.

Solution

$\mathbf{f} = \mathbf{L}\mathbf{z} + \mathbf{m}$ is an affine transformation of $\mathbf{z}$, and we know that Gaussians are closed under affine transformation. Therefore, $\mathbf{f}$ must be a Gaussian distribution.

The mean of $\mathbf{f}$ is given by:

$$\mathbb{E}[\mathbf{f}] = \mathbb{E}[\mathbf{L}\mathbf{z} + \mathbf{m}] \stackrel{(a)}{=} \mathbf{L}\mathbb{E}[\mathbf{z}] + \mathbf{m} \stackrel{(b)}{=} \mathbf{L}\mathbf{0} + \mathbf{m} = \mathbf{m},$$

where we used linearity of expectations in (a) and the fact $\mathbb{E}[\mathbf{z}] = \mathbf{0}$ has zero mean in (b). Similarly, the variance of $\mathbf{f}$ is given by:

$$\mathbb{V}[\mathbf{f}] = \mathbb{V}[\mathbf{L}\mathbf{z} + \mathbf{m}] \stackrel{(c)}{=} \mathbf{L}\mathbb{V}[\mathbf{z}]\mathbf{L}^T \stackrel{(d)}{=} \mathbf{L}\mathbf{I}\mathbf{L}^T = \mathbf{L}\mathbf{L}^T \stackrel{(e)}{=} \mathbf{K},$$

where we used the rules for covariances of affine transformations (e.g. eq (3.5) in Murphy1) in (c), the fact that the covariance matrix of $\mathbf{z}$ is the identity matrix and finally, the Cholesky decomposition itself $\mathbf{K} = \mathbf{L}\mathbf{L}^T$ in (e). Hence, we show that $\mathbf{f} = \mathbf{L}\mathbf{z} + \mathbf{m} \sim \mathcal{N}(\mathbf{m}, \mathbf{K})$.

End of solution

All real-valued symmetric and positive definite matrices have a unique Cholesky decomposition. However, due to finite precision in computers, the covariance matrices used in Gaussian processes can sometimes have negative eigenvalues in practice and hence fail to be positive definite. A common trick is to add a small positive number $\epsilon > 0$ to the diagonal of the covariance matrix, which will increase the eigenvalues of the covariance matrix $\mathbf{K}$ and hence change the sign of a potential small negative eigenvalue. Hence, in practice we construct the covariance matrix as:

$$\mathbf{K}_{nm} = k(\mathbf{x}_n, \mathbf{x}_m) + \epsilon \delta(\mathbf{x}_n - \mathbf{x}_m),$$

where $\delta(\cdot)$ is **Kronecker's delta function** defined by

$$\delta(\mathbf{x}_n - \mathbf{x}_m) = \begin{cases} 1 & \text{if } \mathbf{x}_n = \mathbf{x}_m \\ 0 & \text{otherwise} \end{cases}$$

We often say we add **jitter** to the covariance matrix. ϵ should be as small as positive to avoid affecting the results, e.g. a typical number is $\epsilon \approx 10^{-8}$. If ϵ needs to be significantly bigger, then it is likely that there are some other implementation issues, which should be addressed.

Task 1.2: (optional) Can you prove that adding jitter increases the eigenvalues of the resulting matrix?

Solution

Let $\mathbf{K} \in \mathbb{R}^{N \times N}$ be a symmetric matrix and let $\lambda \in \mathbb{R}$ and $\mathbf{v} \in \mathbb{R}^N$ be an eigenvalue and eigenvector pair of $\mathbf{K}$ such that $\mathbf{K}\mathbf{v} = \lambda\mathbf{v}$. Let $\epsilon > 0$ be a small positive number, then

$$(\mathbf{K} + \epsilon \mathbf{I}_N)\mathbf{v} = \mathbf{K}\mathbf{v} + \epsilon \mathbf{I}_N\mathbf{v} = \lambda\mathbf{v} + \epsilon\mathbf{v} = (\lambda + \epsilon)\mathbf{v}$$

Hence, the corresponding eigenvalue of $\mathbf{K} + \epsilon \mathbf{I}_N$ will be $\lambda + \epsilon$.

End of solution

Task 1.3: Complete the implementation of the function `generate_samples` below for sampling from a multivariate Gaussian using the method described above.

Hints:

- The function `jnp.linalg.cholesky` is handy for computing the Cholesky decomposition of a matrix.
- You need to add the jitter before computing the Cholesky decomposition.

- The argument `key` is for controlling the random number generator in Jax. Sampling is an inherently stochastic procedure, but the explicit use of such keys allows us to build reproducible code in the sense that if we provide the same key, we get the same output.

```
In [2]: def generate_samples(key, m, K, num_samples, jitter=0):
        """ returns M samples from an Gaussian process with mean m and kernel matrix K. The function generates num_samples of  $z \sim N(0, I)$  and transforms them into  $f \sim N(m, K)$  via the Cholesky factorization.

        arguments:
            key          -- jax random key for controlling the random number generator
            m            -- mean vector (shape (N,))
            K            -- kernel matrix (shape NxN)
            num_samples  -- number of samples to generate (positive integer)
            jitter       -- amount of jitter (non-negative scalar)

        returns
            f_samples    -- a numpy matrix containing the samples of f (shape N x num_samples)
        """

        # generate samples from  $N(0, I)$  of shape (N, num_samples)
        zs = random.normal(key, shape=(len(K), num_samples))

        #####
        # Your solution goes here
        #####

        N = len(K)
        L = jnp.linalg.cholesky(K + jitter*jnp.identity(N))
        f_samples = m[:, None] + jnp.dot(L, zs)

        #####
        # End of solution
        #####

        # sanity check of dimensions
        assert f_samples.shape == (len(K), num_samples), f"The shape of f_samples appears wrong. Expected shape ({len(K)}, {num_samples}), but the actual shape was {f_samples.shape}. Please check your code. "
        return f_samples

# sanity check of implementation
key = random.PRNGKey(1)
num_samples = 100000
m = jnp.array([jnp.pi, jnp.sqrt(2)])
V = jnp.array([[0.123, -0.05], [-0.05, 0.123]])
f_samples = generate_samples(key, m, V, num_samples)
assert jnp.linalg.norm(jnp.mean(f_samples, 1) - m) < 1e-2, "The mean of f_samples appears wrong. Please check your code"
assert jnp.linalg.norm(jnp.cov(f_samples) - V) < 1e-2, "The covariance of f_samples appears wrong. Please check your code"
```

We will use this function in the next section to generate samples from a Gaussian process.

Part 2: Stationary covariance functions

Next, we will dive deeper into the covariance functions and their properties. We will focus on so-called **stationary** covariance functions.

The squared exponential kernel is an example of a **stationary** kernel, which means that the covariance function only depends on the difference between two points, i.e. $\tau = \mathbf{x}_n - \mathbf{x}_m$ for two points in the input space $\mathbf{x}_n, \mathbf{x}_m \in \mathbb{R}^D$:

$$k_{\text{SE}}(\mathbf{x}_n, \mathbf{x}_m) = \kappa^2 \exp\left(-\frac{\|\mathbf{x}_n - \mathbf{x}_m\|_2^2}{2\ell^2}\right) = \kappa^2 \exp\left(-\frac{\|\tau\|_2^2}{2\ell^2}\right) = k(\tau).$$

Furthermore, if the covariance function only depends on the distance between any two inputs, i.e. $\|\tau\|_2 = \|\mathbf{x}_n - \mathbf{x}_m\|_2$, then the covariance function is also said to be **isotropic** (isotropic is a fancy word for rotation invariance). For the squared exponential kernel we can write:

$$k_{\text{SE}}(\mathbf{x}_n, \mathbf{x}_m) = k(\|\tau\|) = \kappa^2 \exp\left(-\frac{\|\tau\|_2^2}{2\ell^2}\right), \quad (5)$$

which shows that the squared exponential kernel is both **stationary** and **isotropic** since it can be written as a function $\|\tau\|$. Note that we in the above overload and abuse the notation of k a bit.

Consider the following four covariance functions: the linear kernel, the Matern $\frac{1}{2}$ kernel, the Matern $\frac{3}{2}$ kernel and the so-called automatic relevance determination (ARD) kernel given by

$$\begin{aligned}
k_{\text{Linear}}(\mathbf{x}_n, \mathbf{x}_m) &= \alpha^{-1} \mathbf{x}_m^T \mathbf{x}_n \\
k_{\text{Matern12}}(\mathbf{x}_n, \mathbf{x}_m) &= \kappa^2 \exp\left(-\frac{\|\mathbf{x}_n - \mathbf{x}_m\|}{\ell}\right) \\
k_{\text{Matern32}}(\mathbf{x}_n, \mathbf{x}_m) &= \kappa^2 \left(1 + \frac{\sqrt{3}\|\mathbf{x}_n - \mathbf{x}_m\|}{\ell}\right) \exp\left(-\frac{\sqrt{3}\|\mathbf{x}_n - \mathbf{x}_m\|}{\ell}\right) \\
k_{\text{ARD}}(\mathbf{x}_n, \mathbf{x}_m) &= \kappa^2 \exp\left(-\frac{1}{2} \sum_{i=1}^D \frac{(\mathbf{x}_{n,i} - \mathbf{x}_{m,i})^2}{\ell_i}\right) = \kappa^2 \exp\left(-\frac{1}{2} (\mathbf{x}_n - \mathbf{x}_m)^T \mathbf{L} (\mathbf{x}_n - \mathbf{x}_m)\right),
\end{aligned}$$

where $\mathbf{L} \in \mathbf{R}^{D \times D}$ is a diagonal matrix $\mathbf{L} = \text{diag}(\ell_1^2, \ell_2^2, \dots, \ell_D^2)$. The ARD-kernel can be seen as a generalization of the squared exponential kernel, where each dimension of $\mathbf{x}$ has a separate characteristic lengthscale.

Task 2.1: Which of the four kernels above are stationary kernels and which isotropic kernels?

Solution

The linear kernel is not stationary or isotropic, because there is no way to write the kernel as a function of the difference between the two inputs.

Like the squared exponential, the Matern kernel can be written as a function of the distance between the inputs:

$$k_{\text{Matern12}}(\mathbf{x}_n, \mathbf{x}_m) = k_{\text{Matern12}}(\|\tau\|) = \kappa^2 \exp\left(-\frac{\|\tau\|}{\ell}\right), \quad \tau = \mathbf{x}_n - \mathbf{x}_m$$

and therefore, the Matern $\frac{1}{2}$ is both stationary and isotropic. Similarly, for the Matern32, we can write

$$k_{\text{Matern32}}(\mathbf{x}_n, \mathbf{x}_m) = k_{\text{Matern32}}(\|\tau\|) = \kappa^2 \left(1 + \frac{\sqrt{3}\|\tau\|}{\ell}\right) \exp\left(-\frac{\sqrt{3}\|\tau\|}{\ell}\right), \quad \tau = \mathbf{x}_n - \mathbf{x}_m$$

Finally, for the ARD-kernel we can write

$$k_{\text{ARD}}(\mathbf{x}_n, \mathbf{x}_m) = k_{\text{ARD}}(\tau) = \kappa^2 \exp\left(-\frac{1}{2} \tau^T \mathbf{L} \tau\right), \quad \tau = \mathbf{x}_n - \mathbf{x}_m$$

and therefore, the ARD-kernel is stationary, but not isotropic unless $\mathbf{L} = \ell \cdot \mathbf{I}_D$, in which case it reduces to the squared exponential kernel.

End of solution

Next, we will implement the squared exponential kernel, the Matérn $\frac{1}{2}$, and the Matérn $\frac{3}{2}$ kernel. The and the cell below provide an implementation of these kernels expressed as a function norm of the distance of the inputs, $\|\tau\| = \|\mathbf{x}_n - \mathbf{x}_m\|$, e.g.

$$k_{\text{SE}}(\|\tau\|) = \kappa^2 \exp\left(-\frac{\|\tau\|_2^2}{2\ell^2}\right).$$

```
In [3]: # in the code below tau represents the distance between to input points, i.e. tau = ||x_n - x_m||.
def squared_exponential(tau, kappa, lengthscale):
    return kappa**2*jnp.exp(-0.5*tau**2/lengthscale**2)

def matern12(tau, kappa, lengthscale):
    return kappa**2*jnp.exp(-tau/lengthscale)

def matern32(tau, kappa, lengthscale):
    return kappa**2*(1 + jnp.sqrt(3)*tau/lengthscale)*jnp.exp(-jnp.sqrt(3)*tau/lengthscale)
```

Below you are given a vector $\mathbf{X} \in \mathbb{R}^{N \times 1}$ of $N = 1000$ points on the real line. The points are sorted and equidistantly distributed in the interval $[-6, 6]$.

In order to construct the kernel matrices, all we need to do now is to implement some code for evaluating all the pairwise distances, i.e. $\|\mathbf{x}_n - \mathbf{x}_m\|$ for all pairs of input points, and then feed these distances to the covariance function above along with values for the hyperparameters. This "two-step" implementation makes it easy to switch between different covariance functions.

Task 2.2: Complete the implementation of the class *StationaryIsotropicKernel* below.

Hints: The function `construct_kernel` contain three steps:

1. compute all the pairwise distances between X_1 and X_2 ,
 2. feed these distance to the covariance function `kernel_fun`,
 3. add jitter if needed
- You can either compute the pairwise distances using for-loops or using a vectorized solution. Using for-loops is a bit slower, but might be easier to understand and debug

```
In [4]: class StationaryIsotropicKernel(object):
```

```

def __init__(self, kernel_fun, kappa=1., lengthscale=1.0):
    """
    the argument kernel_fun must be a function of three arguments kernel_fun(|tau|, kappa, lengthscale), e.g.
    squared_exponential = lambda tau, kappa, lengthscale: kappa**2*np.exp(-0.5*tau**2/lengthscale**2)
    """
    self.kernel_fun = kernel_fun
    self.kappa = kappa
    self.lengthscale = lengthscale

def contract_kernel(self, X1, X2, kappa=None, lengthscale=None, jitter=1e-8):
    """ compute and returns the NxM kernel matrix between the two sets of input X1 (shape NxD) and X2 (MxD) using the stationary and isotropic covariance function specified by self.kernel_fun

    arguments:
        X1          -- NxD matrix
        X2          -- MxD matrix
        kappa       -- magnitude (positive scalar)
        lengthscale -- characteristic lengthscale (positive scalar)
        jitter      -- non-negative scalar

    returns
        K          -- NxM matrix
    """

    # extract dimensions
    N, M = X1.shape[0], X2.shape[0]

    # prep hyperparameters
    kappa = self.kappa if kappa is None else kappa
    lengthscale = self.lengthscale if lengthscale is None else lengthscale

    #####
    # Your solution goes here
    #####

    # compute all the pairwise distances efficiently
    dists = jnp.sqrt(jnp.sum((jnp.expand_dims(X1, 1) - jnp.expand_dims(X2, 0))**2, axis=-1))

    # squared exponential covariance function
    K = self.kernel_fun(dists, kappa, lengthscale)

    # add jitter to diagonal for numerical stability
    if len(X1) == len(X2) and jnp.allclose(X1, X2):
        K = K + jitter*jnp.identity(len(X1))

    #####
    # End of solution
    #####

    assert K.shape == (N, M), f"The shape of K appears wrong. Expected shape ({N}, {M}), but the actual shape was {K.shape}. Please check your code. "
    return K

# create an Nx1 vector of equidistant points in [-6, 6]
N = 1000
X = jnp.linspace(-6, 6, N)[:, None]

# hyperparameters
kappa = 1.
scale = 1.

# number of samples to be plotted
num_samples = 10

# specify random seed
seed = 1
key = random.PRNGKey(1)

# instantiate kernel object and construct kernel
kernel = StationaryIsotropicKernel(squared_exponential, kappa, scale)
K = kernel.contract_kernel(X, X)

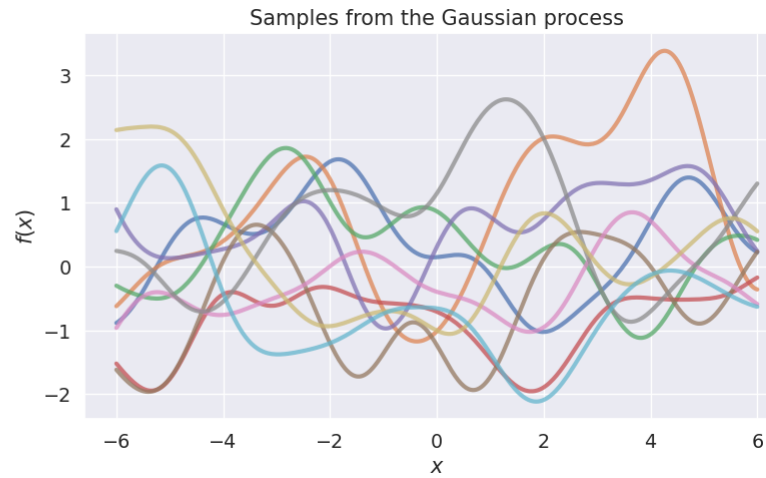
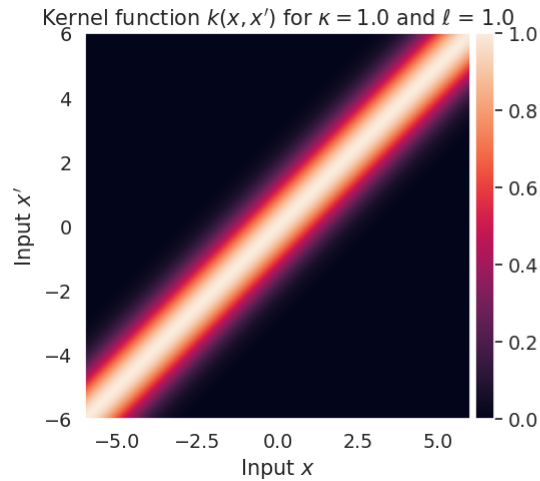
# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 5))

m = jnp.zeros(len(X))
im = ax[0].pcolormesh(X.flatten(), X.flatten(), K, shading='auto')
ax[0].set(xlabel='Input $x$', ylabel='Input $x$ $x$', title=f'Kernel function $k(x, x')$ for $\kappa = \{kappa:2.1f\}$ and $\ell = \{scale:2.1f\}$')
ax[0].grid(False)
ax[0].set_aspect('equal')
add_colorbar(im, fig, ax[0])

f_samples = generate_samples(key, m, K, num_samples=num_samples, jitter=1e-8)

```

```
ax[1].plot(X, f_samples, alpha=0.75, linewidth=3);
ax[1].grid(True)
ax[1].set(xlabel='$x$', ylabel='$f(x)$', title='Samples from the Gaussian process');
```



If your code is correct, the cell above you produce a plot of the covariance matrix as well as a plotting a number of samples form the corresponding Gaussian process.

Task 2.3: Inspect the plots above. Change the parameters κ , ℓ and explain how they affect the structure of the kernel (left panel) [Discussion question]

Task 2.4: Inspect the plots above. Change the parameters κ , ℓ and explain how they affect the prior samples (right panel) [Discussion question]

Task 2.5: Generate samples from the Matern $\frac{1}{2}$ and Matern $\frac{3}{2}$ kernels. How do they compare to samples from a squared exponential [Discussion question]

Solution

```
In [5]: kernel_dict = {'Squared exponential': squared_exponential,
                    'Matern12': matern12,
                    'Matern32': matern32}

# hyperparameters
kappa = 1.
scale = 1.

# num_samples
num_samples = 5

# specify random seed
seed = 1
key = random.PRNGKey(1)

# plot
fig, ax = plt.subplots(1, 3, figsize=(20, 4))

for i, (name, cov_fun) in enumerate(kernel_dict.items()):
    K = StationaryIsotropicKernel(cov_fun, kappa, scale).construct_kernel(X, X)
    f_samples = generate_samples(key, m, K, num_samples=num_samples, jitter=1e-8)
    ax[i].plot(X, f_samples, alpha=0.75, linewidth=3);
    ax[i].set(xlabel='$x$', ylabel='$f(x)$', title=name)
```

**End of solution**

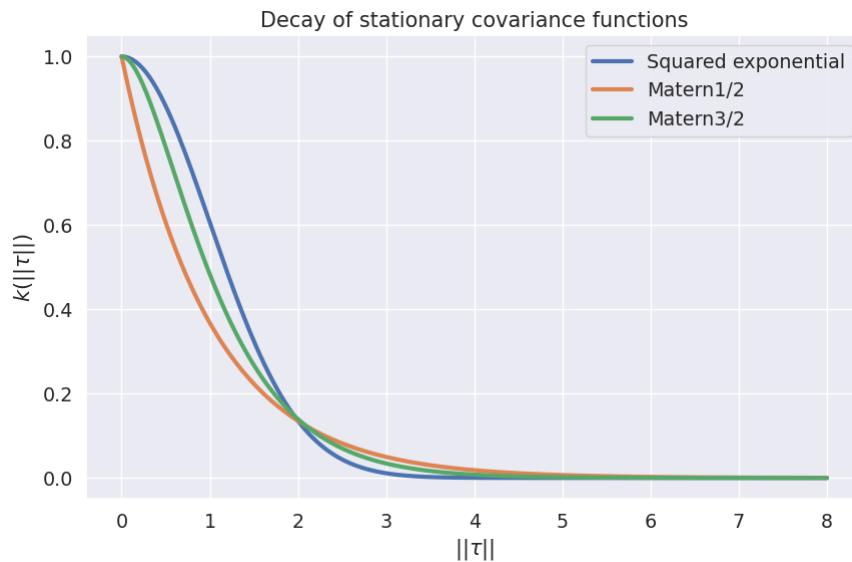
A simple way to understand and compare the three covariance functions (squared exponential, Matérn12, and Matérn32) is to study how they decay as a function of $\|\tau\| = \|\mathbf{x}_n - \mathbf{x}_m\|$:

```
In [6]: tau = jnp.linspace(0, 8, 200)

kappa = 1
scale = 1

fig, ax = plt.subplots(1, 1, figsize=(10, 6))
ax.plot(tau, squared_exponential(tau, kappa=kappa, lengthscale=scale), linewidth=3, label='Squared exponential')
ax.plot(tau, matern12(tau, kappa=kappa, lengthscale=scale), linewidth=3, label='Matern1/2')
ax.plot(tau, matern32(tau, kappa=kappa, lengthscale=scale), linewidth=3, label='Matern3/2')
ax.set(xlabel='$\|\tau\|$', ylabel='$k(\|\tau\|)$', title='Decay of stationary covariance functions')
ax.legend()
```

Out[6]: <matplotlib.legend.Legend at 0x7f96e0480c80>



Suppose the distance between two input points $\mathbf{x}_n$ and $\mathbf{x}_m$ is $d \cdot \ell$, i.e. $\|\tau\| = \|\mathbf{x}_n - \mathbf{x}_m\| = d \cdot \ell$.

Task 2.6: Using the squared exponential covariance function and assume $\kappa = 1$, what is the covariance between the function values $f(\mathbf{x}_n)$ and $f(\mathbf{x}_m)$ for $d = 0, 1, 2, 3, 4, 5$?

Hints: Use eq. (5)

Solution

For $\kappa = 1$ and $\|\tau\| = d \cdot \ell$, we have

$$\begin{aligned} k(\mathbf{x}_n, \mathbf{x}_m) &= k(\|\tau\|) \\ &= \kappa^2 \exp\left(-\frac{\|\tau\|^2}{2\ell^2}\right) \\ &= \kappa^2 \exp\left(-\frac{(d \cdot \ell)^2}{2\ell^2}\right) \\ &= \exp\left(-\frac{d^2 \ell^2}{2\ell^2}\right) \\ &= \exp\left(-\frac{d^2}{2}\right) \end{aligned}$$

For $d = 0, \dots, 5$, this becomes:

```
In [7]: for d in jnp.arange(6):
        print('d = %d: %5.4f' % (d, jnp.exp(-d**2/2)))

d = 0: 1.0000
d = 1: 0.6065
d = 2: 0.1353
d = 3: 0.0111
d = 4: 0.0003
d = 5: 0.0000
```

Task 2.7: What can we say about the statistical relationship between the function values $f(\mathbf{x}_n)$ and $f(\mathbf{x}_m)$ when $d \gg 4$? What can we say in general about the function values $f(\mathbf{x}_n)$ and $f(\mathbf{x}_m)$ for points $\mathbf{x}_n$ and $\mathbf{x}_m$ that separated by more than a few lengthscales?

Solution

For $d \gg 4$, the covariance between the function values $f(\mathbf{x}_n)$ and $f(\mathbf{x}_m)$ is smaller than $\exp(-8) \approx 10^{-3}$. Hence, the function values are effectively *uncorrelated*. Moreover, jointly Gaussian random variables have the special property that if uncorrelated variables are also **statistically independent**. Hence, function values for input points separated by more than a few lengthscales are effective statistically independent a priori for the squared exponential kernel.

End of solution

Part 3: Non-linear regression using Gaussian processes

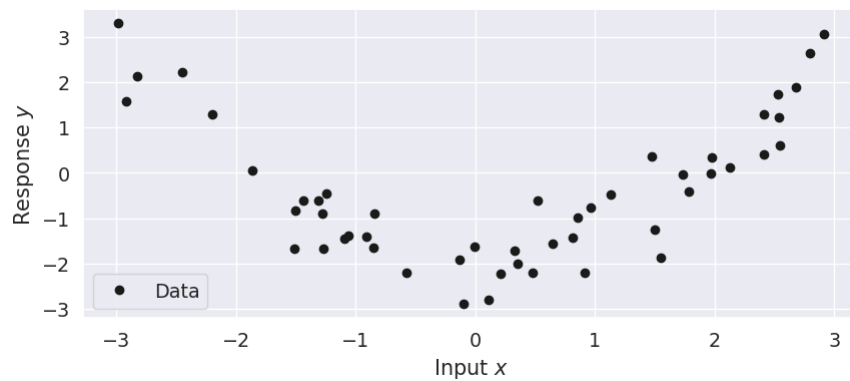
In this part, we will study how Gaussian processes can be used for non-linear regression. First, we will study a simple toy data set $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ for $N = 50$:

```
In [8]: # load data
data = jnp.load('./data_exercise5.npz')
N = data['N']
Xtrain, ytrain = data['X'], data['y']

# for predictions
Xstar = jnp.linspace(-3, 9, 100)[:, None]

# function for plotting
def plot_data(ax):
    ax.plot(Xtrain, ytrain, 'k.', markersize=12, label='Data')
    ax.grid(True)
    ax.set_xlabel('Input $x$')
    ax.set_ylabel('Response $y$')
    ax.legend()

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
plot_data(ax)
```

We will adopt the following model

$$y_n = f(\mathbf{x}_n) + e_n,$$

where $f(\mathbf{x})$ is assumed to be a Gaussian process. Assuming the noise e_n is i.i.d. and Gaussian, the joint model for the training data becomes:

$$p(\mathbf{y}, \mathbf{f}) = \mathcal{N}(\mathbf{y} | \mathbf{f}, \sigma^2 \mathbf{I}) \mathcal{N}(\mathbf{f} | \mathbf{0}, \mathbf{K})$$

where $\mathbf{K}_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$ is the covariance matrix. We will also use the squared exponential and Matérn covariance functions in this part. Thus, this model (when using these covariance functions) has three hyperparameters in total: $\theta = \{\sigma, \kappa, \ell\}$, where

- $\sigma > 0$ is the standard deviation of the noise,
- $\kappa > 0$ is the magnitude of the kernel,
- $\ell > 0$ is the lengthscale of the kernel.

We will use $\theta_K = \{\kappa, \ell\}$ to denote the hyperparameters of the kernel K and θ to denote all hyperparameters of the model.

Below you are given an incomplete implementation of a class for Gaussian process regression. The function called **compute_posterior** is supposed to the posterior mean and covariance matrix of the function values $f(\mathbf{x}^*)$ for input $\mathbf{x}^*$ conditioned on the training data $\mathbf{y}$. The class contains a template for the following functions:

- `predict_f` : computes the posterior distribution of the function value $f(\mathbf{x}^*)$ for a new input, $\mathbf{x}^*$, i.e. $p(f^* | \mathbf{y}) = \mathcal{N}(f^* | \mu_{f^*}, \Sigma_{f^*})$
- `predict_y` : computes the posterior predictive distribution of for a new observation at new input, $\mathbf{x}^*$, i.e. $p(y^* | \mathbf{y}) = \mathcal{N}(y^* | \mu_{y^*}, \Sigma_{y^*})$
- `posterior_samples` : generates samples from the posterior distribution of f^* for a set of inputs, i.e. $\mathbf{f}^* \sim p(f^* | \mathbf{y})$
- `log_marginal_likelihood` : evaluate the log marginal likelihood for a given set of hyperparameters θ

In this part, we will focus on the first three and then come back to the marginal likelihood later.

Task 3.1: Complete the implementation of the function `predict_f` below.

Hints:

- Look at eq. (17.34), (17.35) and (17.36) in Murphy1
- For computing matrix-vector products of the form $\mathbf{A}^{-1}\mathbf{x}$, `np.linalg.solve(A, x)` is numerically much more stable than `np.linalg.inv(A)@x`.
- If you prefer, you can move the class to a separate python module and import it to the notebook if you prefer not the scroll up and down during as you make it through the notebook.

Task 3.2: Complete the implementation of the function `predict_y` below.

Hints: Use the function `predict_f` you implemented in the previous task to do most of the work

Task 3.3: Complete the implementation of the function `posterior_samples` below.

Hints: The implementation of this should be easy and make heavy use of functions you already implemented

```
In [ ]:

In [9]: def plot_with_uncertainty(ax, Xp, gp, color='r', color_samples='b', title="", num_samples=0, seed=0):

    mu, Sigma = gp.predict_y(Xp)
    mean = mu.ravel()
    std = jnp.sqrt(jnp.diag(Sigma))
```

```

# random seed
key = random.PRNGKey(seed)

# plot distribution
ax.plot(Xp, mean, color=color, label='Mean')
ax.plot(Xp, mean + 2*std, color=color, linestyle='--')
ax.plot(Xp, mean - 2*std, color=color, linestyle='--')
ax.fill_between(Xp.ravel(), mean - 2*std, mean + 2*std, color=color, alpha=0.25, label='95% interval')

# generate samples
if num_samples > 0:
    fs = gp.posterior_samples(key, Xstar, num_samples)
    ax.plot(Xp, fs[:,0], color=color_samples, alpha=.25, label="$f(x)$ samples")
    ax.plot(Xp, fs[:, 1:], color=color_samples, alpha=.25)

ax.set_title(title)

class GaussianProcessRegression(object):

    def __init__(self, X, y, kernel, kappa=1., lengthscale=1., sigma=1/2, jitter=1e-8):
        """
        Arguments:
            X          -- NxD input points
            y          -- Nx1 observed values
            kernel     -- must be instance of the StationaryIsotropicKernel class
            jitter     -- non-negative scaler
            kappa      -- magnitude (positive scalar)
            lengthscale -- characteristic lengthscale (positive scalar)
            sigma      -- noise std. dev. (positive scalar)
        """
        self.X = X
        self.y = y
        self.N = len(X)
        self.kernel = kernel
        self.jitter = jitter
        self.set_hyperparameters(kappa, lengthscale, sigma)
        self.check_dimensions()

    def check_dimensions(self):
        assert self.X.ndim == 2, f"The variable X must be of shape (N, D), however, the current shape is: {self.X.shape}"
        N, D = self.X.shape

        assert self.y.ndim == 2, f"The variabel y must be of shape (N, 1), however. the current shape is: {self.y.shape}"
        assert self.y.shape == (N, 1), f"The variabel y must be of shape (N, 1), however. the current shape is: {self.y.shape}"

    def set_hyperparameters(self, kappa, lengthscale, sigma):
        self.kappa = kappa
        self.lengthscale = lengthscale
        self.sigma = sigma

    def posterior_samples(self, key, Xstar, num_samples):
        """
        generate samples from the posterior  $p(f^*|y, x^*)$  for each of the inputs in Xstar

        Arguments:
            key          -- jax random key for controlling the random number generator
            Xstar       -- PxD prediction points

        returns:
            f_samples    -- numpy array of (P, num_samples) containing num_samples for each of the P inputs in Xstar
        """
        #####
        # Your solution goes here
        #####

        mu, Sigma = self.predict_f(Xstar)
        f_samples = generate_samples(key, mu.ravel(), Sigma, num_samples)

        #####
        # End of solution
        #####

        assert (f_samples.shape == (len(Xstar), num_samples)), f"The shape of the posterior mu seems wrong. Expected ({len(Xstar)}, {num_samples}), but actual shape was {f_samples.shape}. Please check implementation"
        return f_samples

    def predict_y(self, Xstar):
        """ returns the posterior distribution of  $y^*$  evaluated at each of the points in  $x^*$  conditioned on (X, y)

```

```

Arguments:
Xstar          -- PxD prediction points

returns:
mu             -- Px1 mean vector
Sigma          -- PxP covariance matrix
"""

#####
# Your solution goes here
#####

# prepare relevant matrices
mu, Sigma = self.predict_f(Xstar)
Sigma = Sigma + self.sigma**2 * jnp.identity(len(mu))

#####
# End of solution
#####

return mu, Sigma

def predict_f(self, Xstar):
    """ returns the posterior distribution of f^* evaluated at each of the points in x^* conditioned on (X, y)

    Arguments:
    Xstar          -- PxD prediction points

    returns:
    mu             -- Px1 mean vector
    Sigma          -- PxP covariance matrix
    """

    #####
    # Your solution goes here
    #####

    # prepare relevant matrices
    k = self.kernel.construct_kernel(Xstar, self.X, self.kappa, self.lengthscale, jitter=self.jitter)
    K = self.kernel.construct_kernel(self.X, self.X, self.kappa, self.lengthscale, jitter=self.jitter)
    Kstar = self.kernel.construct_kernel(Xstar, Xstar, self.kappa, self.lengthscale, jitter=self.jitter)

    # Compute C matrix
    C = K + self.sigma**2*jnp.identity(len(self.X))

    # computer mean and Sigma
    mu = jnp.dot(k, jnp.linalg.solve(C, self.y))
    Sigma = Kstar - jnp.dot(k, jnp.linalg.solve(C, k.T))

    #####
    # End of solution
    #####

    # sanity check for dimensions
    assert (mu.shape == (len(Xstar), 1)), f"The shape of the posterior mu seems wrong. Expected ({len(Xstar)}, 1), but actual shape was {mu.shape}. Please check implementation"
    assert (Sigma.shape == (len(Xstar), len(Xstar))), f"The shape of the posterior Sigma seems wrong. Expected ({len(Xstar)}, {len(Xstar)}), but actual shape was {Sigma.shape}. Please check implementation"

    return mu, Sigma

def log_marginal_likelihood(self, kappa, lengthscale, sigma):
    """
        evaluate the log marginal likelihood p(y) given the hyperparaemters

    Arguments:
    kappa          -- positive scalar
    lengthscale    -- positive scalar
    sigma          -- positive scalar
    """

    #####
    # Your solution goes here
    #####

    # prepare kernels
    K = self.kernel.construct_kernel(self.X, self.X, kappa, lengthscale)
    C = K + sigma**2*jnp.identity(self.N)

    # compute Cholesky decomposition
    L = jnp.linalg.cholesky(C)
    v = jnp.linalg.solve(L, self.y)

    # compute log marginal likelihood

```

```

logdet_term = jnp.sum(jnp.log(jnp.diag(L)))
quad_term = 0.5*jnp.sum(v**2)
const_term = -0.5*self.N*jnp.log(2*jnp.pi)

return const_term - logdet_term - quad_term

#####
# End of solution
#####

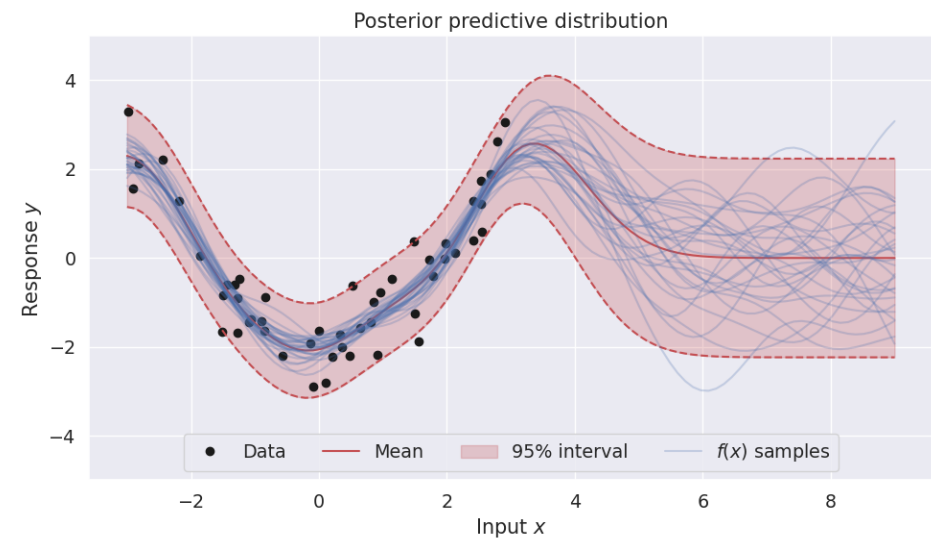
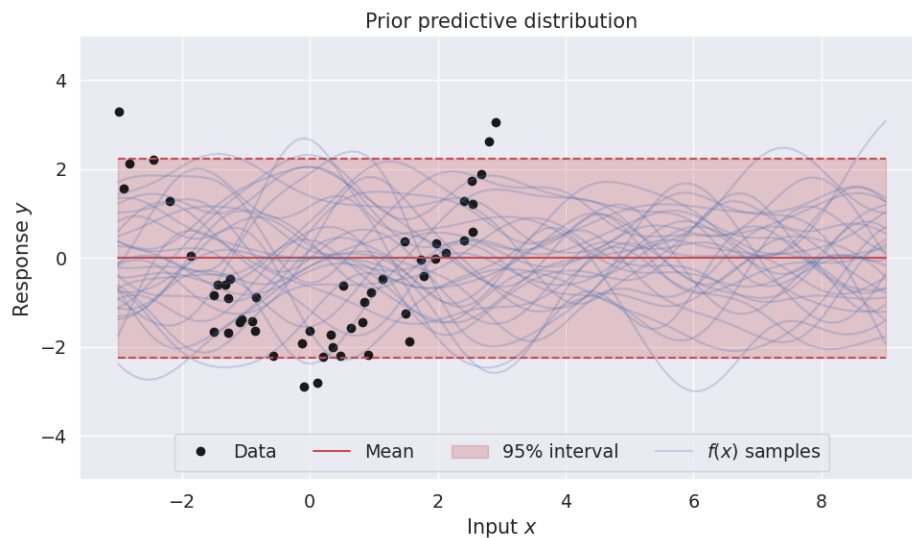
# instantiate kernel
kernel = StationaryIsotropicKernel(kernel_fun=squared_exponential)

# instantiate GP without data (hence, posterior=prior) and with data
gp_prior = GaussianProcessRegression(jnp.zeros((0, 1)), jnp.zeros((0, 1)), kernel)
gp_post = GaussianProcessRegression(Xtrain, ytrain, kernel)

# plot
fig, ax = plt.subplots(1, 2, figsize=(25, 6))
plot_data(ax[0])
plot_with_uncertainty(ax[0], Xstar, gp_prior, title='Prior predictive distribution', num_samples=30)
ax[0].legend(loc='lower center', ncol=4)
ax[0].set_ylim((-5, 5))

plot_data(ax[1])
plot_with_uncertainty(ax[1], Xstar, gp_post, title='Posterior predictive distribution', num_samples=30)
ax[1].legend(loc='lower center', ncol=4)
ax[1].set_ylim((-5, 5));

```



If the code has been implemented correct, then the left figure above shows 30 samples from the prior predictive distribution along with the mean and 95% interval from the prior predictive distribution, whereas the right figure shows the same quantities for the posterior predictive distribution.

Task 3.4: Inspect the plots above. How does the prior and posterior predictive differ in: a) regions close to the data points? b) in regions far from the data point? [Discussion question]

Task 3.5: Explain this behavior using the equations for the posterior mean and covariance of f^* (and what you saw in part 2)? [Discussion question]

Task 3.6: Visualize the prior covariance matrix for f^* and posterior covariance for f^* matrix side-by-side as images and comment on what you see

Solution

```

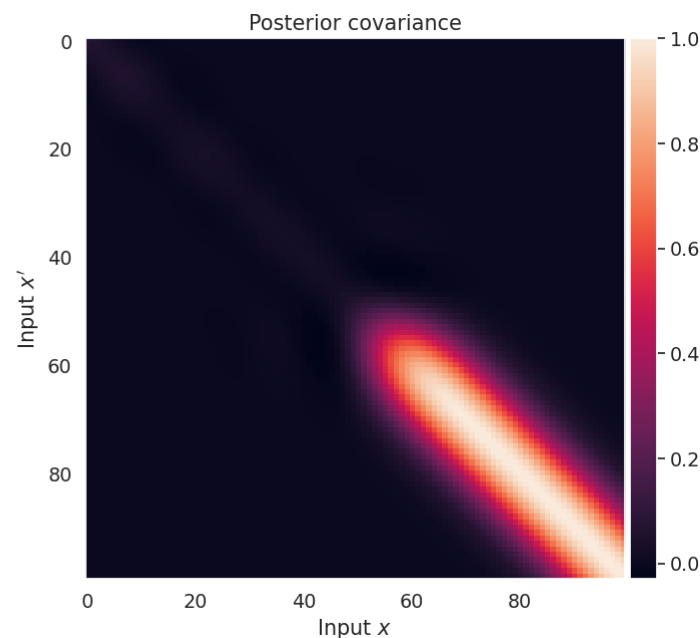
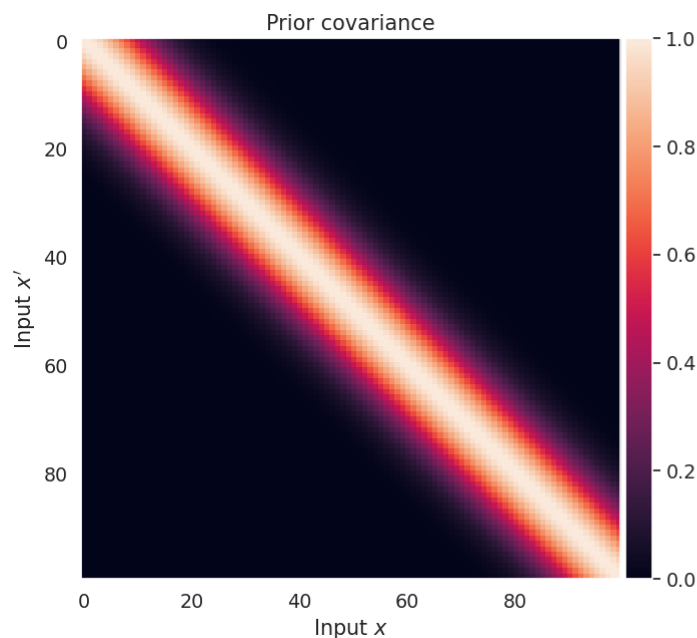
In [10]: ## compute mean and covariance of prior and posterior for f
mu_prior, Sigma_prior = gp_prior.predict_f(Xstar)
mu_post, Sigma_post = gp_post.predict_f(Xstar)

# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 7))
im = ax[0].imshow(Sigma_prior)

```

```
ax[0].grid(False)
ax[0].set(xlabel='Input $x$', ylabel='Input $x$', title='Prior covariance')
add_colorbar(im, fig, ax[0])

im = ax[1].imshow(Sigma_post)
ax[1].grid(False)
ax[1].set(xlabel='Input $x$', ylabel='Input $x$', title='Posterior covariance');
add_colorbar(im, fig, ax[1])
```



End of solution

Task 3.7: Change the values of kappa and scale and explain how it affects the prior and posterior. For example, what happens if you choose the lengthscale ℓ to be either 0.25, 1, or 5 while keeping $\kappa = 1$ and $\sigma = \frac{1}{2}$? Do similar experiments with σ and κ . [Discussion question]

Task 3.8: Experiment with the two other kernels, the Matérn $\frac{1}{2}$ and the Matérn $\frac{3}{2}$. How does that affect the fits and the predictions? [Discussion question]

Part 4: Hyperparameter optimization using the marginal likelihood

Before we can apply this model in practice, we will study how we can use the **marginal likelihood** to estimate the hyperparameters of the model. If we let $\theta \in \mathbb{R}^K$ denote all of our hyperparameters, then the **evidence approximation** suggests that we can estimate θ as follows

$$\hat{\theta} = \arg \max_{\theta} \ln p(\mathbf{y}|\theta),$$

where $p(\mathbf{y}|\theta)$ is the marginal likelihood of the model (see eq. (17.52) in Murphy1).

Task 4.1: Go back up to the implementation of `GaussianProcessRegression` -class and complete the implementation of the function `log_marginal_likelihood`

- Hints: The expression for the marginal likelihood contains both determinants and matrix inverse, which can be numerically fragile and lead to underflow/overflow. Hence, you need to be careful with the numerical implementation. See the details in the slides for this week

If your implementation is correct, then the code below implement an optimization routine for the marginal likelihood. Note that we re-parametrize the hyperparameters in the log-domain before optimizing to ensure that the hyperparameter remain positive. We rely on the package `autograd` for computing the gradients.

```
In [11]: def optimize_hyperparameters(gp, theta_init):

    # define optimization objective as the negative log marginal likelihood
    objective = lambda params: -gp.log_marginal_likelihood(jnp.exp(params[0]), jnp.exp(params[1]), jnp.exp(params[2]))

    # optimize using gradients
    res = minimize(value_and_grad(objective), jnp.log(theta_init), jac=True)
```

```

# check for success
if not res.success:
    print('Warning: optimization failed!')

# return resultss
theta = jnp.exp(res.x)
return theta

```

Let's run it on our toy dataset:

```

In [12]: kernel = StationaryIsotropicKernel(kernel_fun=squared_exponential)
gp_prior = GaussianProcessRegression(jnp.zeros((0, 1)), jnp.zeros((0, 1)), kernel)
gp_post = GaussianProcessRegression(Xtrain, ytrain, kernel)

# optimize
theta_hat = optimize_hyperparameters(gp_post, theta_init=jnp.array([1,1,1]))

# extract individual hyperparameters
kappa_hat, scale_hat, sigma_hat = theta_hat

# print
print('Estimated hyperparameters')
print(f'\tsigma: {sigma_hat:3.2f}')
print(f'\tkappa: {kappa_hat:3.2f}')
print(f'\tscale: {scale_hat:3.2f}')

gp_prior = GaussianProcessRegression(jnp.zeros((0, 1)), jnp.zeros((0, 1)), kernel)
gp_post = GaussianProcessRegression(Xtrain, ytrain, kernel)
gp_prior.set_hyperparameters(kappa_hat, scale_hat, sigma_hat)
gp_post.set_hyperparameters(kappa_hat, scale_hat, sigma_hat)

mu_prior, Sigma_prior = gp_prior.predict_f(Xstar)
mu_post, Sigma_post = gp_post.predict_f(Xstar)

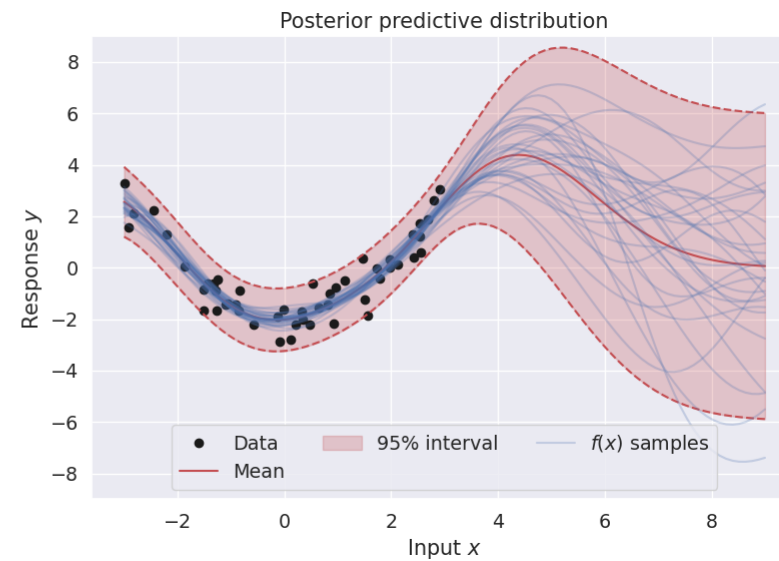
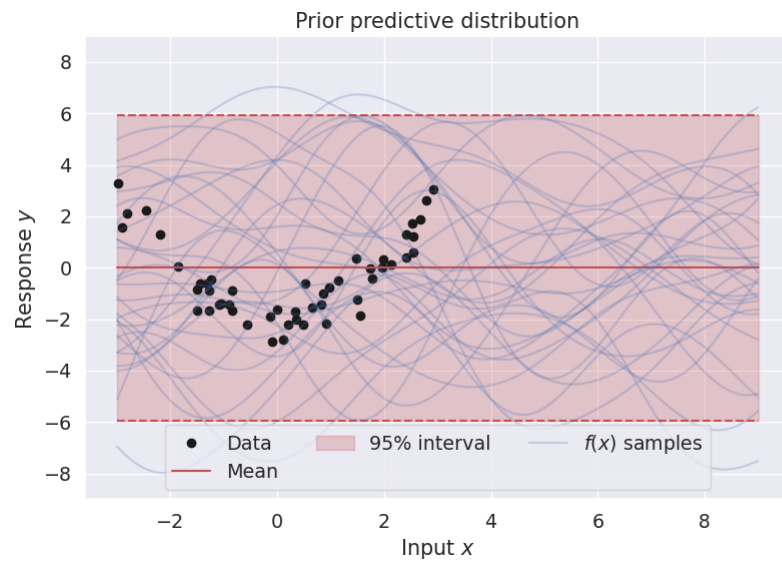
# plot
fig, ax = plt.subplots(1, 2, figsize=(20, 6))
plot_data(ax[0])
plot_with_uncertainty(ax[0], Xstar, gp_prior, title='Prior predictive distribution', num_samples=30)
ax[0].legend(loc='lower center', ncol=3)
ax[0].set_ylim((-9, 9))
plot_data(ax[1])
plot_with_uncertainty(ax[1], Xstar, gp_post, title='Posterior predictive distribution', num_samples=30)
ax[1].legend(loc='lower center', ncol=3)
ax[1].set_ylim((-9, 9));

```

```

Estimated hyperparameters
sigma: 0.59
kappa: 2.92
scale: 1.82

```



Task 4.2: Comment on the quality of the fit. Does it seem reasonable? [Discussion question]

Task 4.3: If we were to fit the three hyperparameters using 5-fold cross-validation and a grid search rather than using the marginal likelihood, how many times would we have to train the model? Assume we would 10 different values for each hyperparameter.

Solution

For each fold, we would have to compute $10^3 = 1000$ different parameter combinations and thus, we would have to train the model $5 \cdot 1000 = 5000$ times in total.

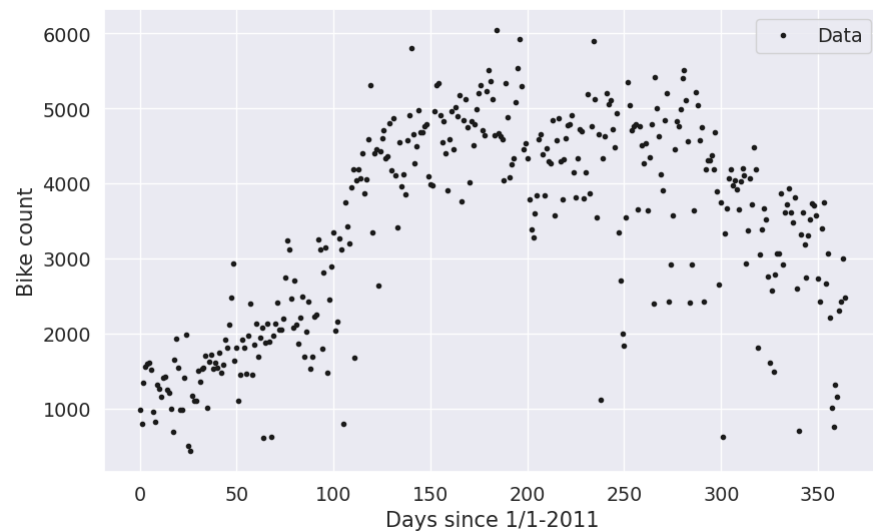
End of solution

Part 5: Analysing the bike sharing data set

In this section, your task is to fit a GP regression model (with squared exponential kernel) to a subset of the bike sharing dataset (Source: <https://ride.capitalbikeshare.com/system-data>). The dataset consists of the number of rented bikes in 2011.

```
In [13]: data = jnp.load('./data_exercise5b.npz')
day = data['day']
bike_count = data['bike_count']

fig, ax = plt.subplots(1, 1, figsize=(10, 6))
ax.plot(day, bike_count, 'k.', label='Data')
ax.legend()
ax.set(xlabel='Days since 1/1-2011', ylabel='Bike count');
```



Task 5.1: Log-transform the bike counts, remove the mean and scale to unit variance. Plot the preprocessed data set.

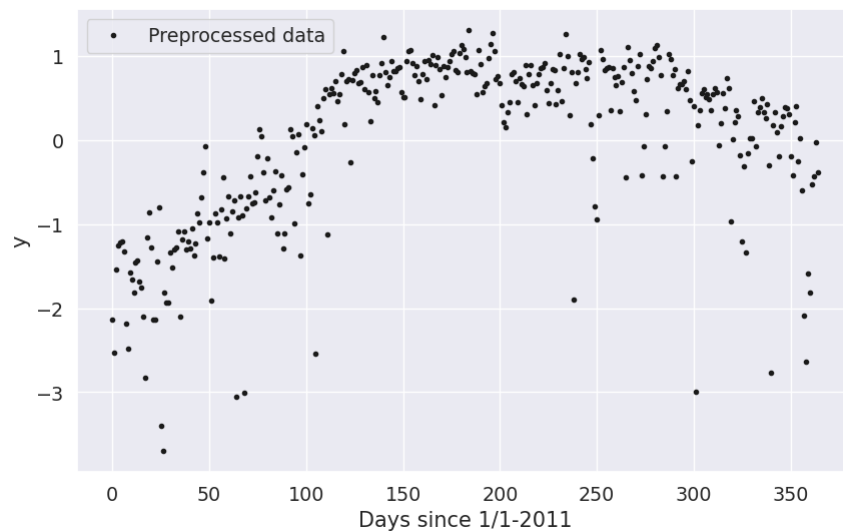
Solution

```
In [14]: # log-transform
y = jnp.log(bike_count)

# remove mean and scale to unit variance
ym, ys = jnp.mean(y), jnp.std(y)
y = (y-ym)/ys

X = day

# plot result
fig, ax = plt.subplots(1, 1, figsize=(10, 6))
ax.plot(X, y, 'k.', label='Preprocessed data')
ax.legend()
ax.set(xlabel='Days since 1/1-2011', ylabel='y');
```

End of solution

Task 5.2: Estimate the hyperparameters of a Gaussian process regression model with a squared exponential kernel from the preprocessed data. Report the estimated values of each hyperparameter.

Solution

```
In [15]: # prep gp
gp = GaussianProcessRegression(X, y, StationaryIsotropicKernel(squared_exponential))

# optimize hyperparameters
theta_hat = optimize_hyperparameters(gp, theta_init=jnp.array([1, 1, 50.]))
kappa_hat, scale_hat, sigma_hat = theta_hat

# use new hyperparameters
gp.set_hyperparameters(kappa_hat, scale_hat, sigma_hat)

print('Estimated hyperparameters')
print(f'\tsigma: {sigma_hat:3.2f}')
print(f'\tkappa: {kappa_hat:3.2f}')
print(f'\tscale: {scale_hat:3.2f}')
print('')
print(f'Log Z: {gp.log_marginal_likelihood(kappa_hat, scale_hat, sigma_hat):3.2f}')

Estimated hyperparameters
  sigma: 0.57
  kappa: 1.08
  scale: 78.63
```

Log Z: -324.85

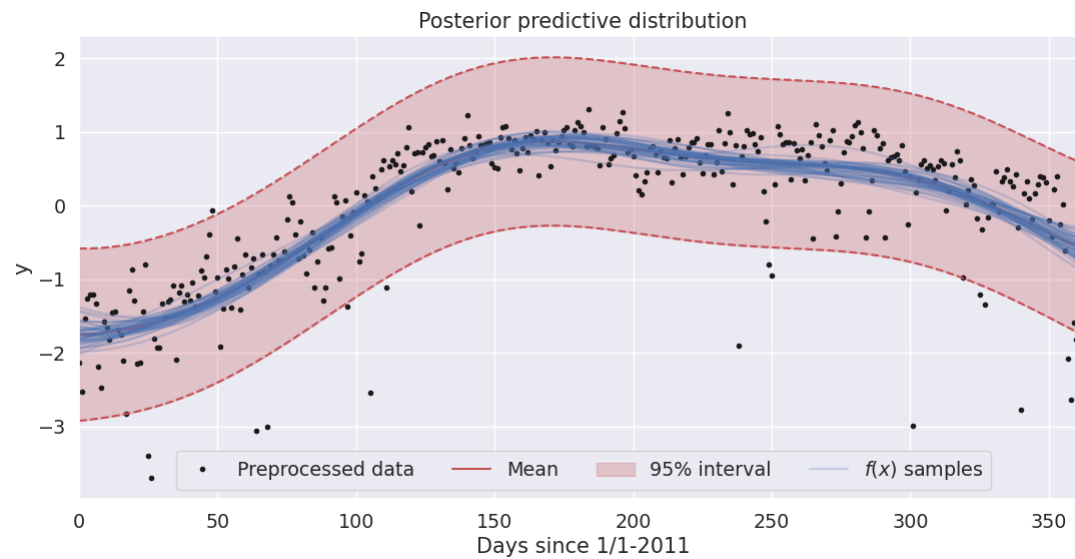
Task 5.3: Compute and visualize the predictive distribution for the interval $\text{day} \in \{0, 1, 2, \dots, 370\}$.

```
In [16]: # days for predictions
Xstar = jnp.arange(0, 370)[: , None]

# compute posterior of y for all days in interval [0, 370]
mu_post, Sigma_post = gp.predict_f(Xstar)

# visualize
fig, ax = plt.subplots(1, 1, figsize=(13, 6))
ax.plot(X, y, 'k.', label='Preprocessed data')
plot_with_uncertainty(ax, Xstar, gp, title='Posterior predictive distribution', num_samples=50)
ax.legend(loc='lower center', ncol=4)
ax.set(xlabel='Days since 1/1-2011', ylabel='y')
ax.set_xlim(X.min(), X.max())
```

Out[16]: (0.0, 364.0)



Task 5.4: The observation model $p(y|f) = \mathcal{N}(y(x)|f(x), \sigma^2)$ assumes the additive noise is **isotropic** and **identically distributed** for all observations. Is that a reasonable assumption for this data set? **[Discussion question]**

Task 5.5: Generate 50 samples from the posterior in the interval $[0, 370]$, map them back to the original measurement space and plot them on top of the original dataset

Solution

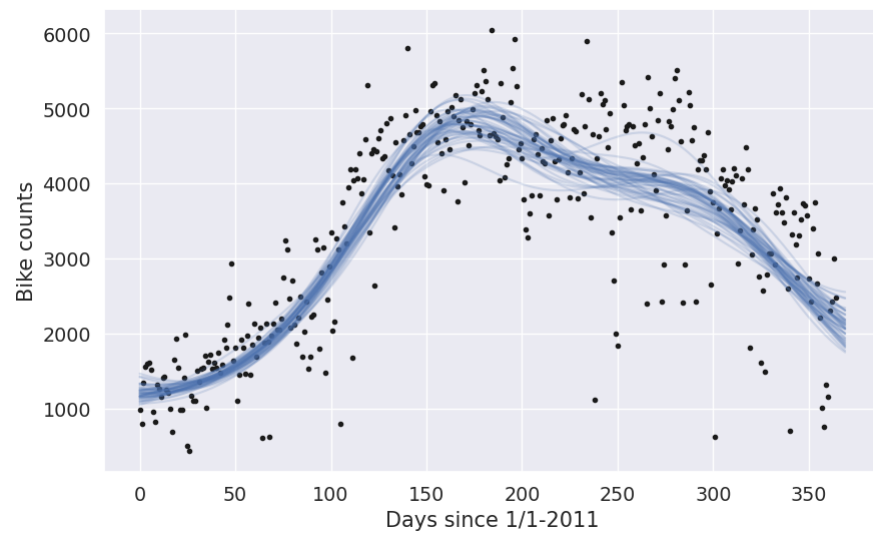
```
In [17]: # specify random seed
seed = 0
key = random.PRNGKey(seed)

# generate samples
f = gp.posterior_samples(key, Xstar, num_samples=50)

# compute "reverse standardization" for each sample
f2 = ys*f + ym

# exponentiate
f3 = jnp.exp(f2)

# plot
fig, ax = plt.subplots(1, 1, figsize=(10, 6))
ax.plot(X, bike_count, 'k.', label='Preprocessed data')
ax.plot(Xstar, f3, 'b', alpha=0.2);
ax.set(xlabel='Days since 1/1-2011', ylabel='Bike counts');
```



End of solution